

Fixing the Spark Notebook

From Timeouts to Tuned Configurations

Data Engineering Course

September 13, 2026

Outline

- 1 The Problem: Notebook Timeouts
- 2 How We Got the Idea: Old Notebooks
- 3 Self-Configured “Large” Notebooks
- 4 Approach: Fix the Notebook

The Problem: Notebook Timeouts

Symptoms

- Running the Spark notebook **as-is** takes **4–6 hours**.
- Eventually it **times out** or loses track.
- Not exactly a crash — it silently hangs after several hours.
- Hard to see what is happening at that point.

Impact

- No useful output after a full working day.
- Iteration is painfully slow.
- We need a more reliable and faster execution.

Why Does It Time Out?

- The notebook processes a **large amount of data**.
- Default Spark configuration may not provide enough:
 - Executor memory
 - Number of executors
 - Driver memory
 - CPU cores
- When resources are insufficient, Spark spills to disk, retries, and slows down dramatically.
- After 5–6 hours, the session may be killed by the cluster manager or simply become unresponsive.

Key Insight

We need to **change the configuration** — executor memory, number of executors, and possibly driver memory — to match the workload.

Learning from Old Notebooks

- We examined **3–5 year old notebooks** in the repository.
- Those notebooks were **manually configuring** Spark:
 - `spark.executor.memory`
 - `spark.executor.cores`
 - `spark.executor.instances`
 - `spark.driver.memory`
- They were designed to run **large workloads** and completed successfully.
- This gave us the idea: **replicate their configuration approach**.

Observation

Older code did not have the luxury of pre-configured “large” notebooks. Engineers explicitly tuned Spark for each job.

Example: Old-Style Manual Configuration

```
1 from pyspark.sql import SparkSession
2
3 spark = (SparkSession.builder
4         .appName("LargeNotebookJob")
5         .config("spark.executor.memory", "16g")
6         .config("spark.executor.cores", "4")
7         .config("spark.executor.instances", "10")
8         .config("spark.driver.memory", "8g")
9         .config("spark.sql.shuffle.partitions", "200")
0         .getOrCreate())
```

Listing 1: Manual Spark configuration in an old notebook

- Explicit values for memory, cores, and instances.
- Tuned for a specific large workload.
- This is what we might need to do — or use a pre-configured notebook.

Self-Configured Large Notebooks

What are they?

Notebooks that come with **pre-set Spark configurations** optimized for large workloads.

- The **tech team** creates different kinds of notebooks:
 - Small / default notebooks for quick exploration.
 - Medium notebooks for standard jobs.
 - **Large notebooks** for heavy processing.
- These notebooks are **self-configured** — you do not need to manually set memory or executors.
- They are ready to use when you know your job is resource-intensive.

Recommendation

Use a **large self-configured notebook** when your workload is known to be heavy.
Fall back to manual configuration only if needed.

How to Identify a Large Notebook

- Check the notebook metadata or header comments.
- Look for pre-set Spark configuration cells:

```
1 # This notebook is pre-configured for LARGE workloads.  
2 # Executor memory: 32g, Executors: 20, Cores: 8  
3 spark = SparkSession.builder.getOrCreate()  
4 # Configuration is applied automatically by the environment.
```

Listing 2: Self-configured large notebook (example)

- Ask the tech team which notebook profile to use.
- In many clusters, you can select the notebook type from a dropdown.

Step-by-Step Plan

- 1 **Reproduce the problem** — run as-is and confirm the timeout.
- 2 **Inspect old notebooks** — find manual configs that worked.
- 3 **Check available notebook profiles** — ask tech team or look for large notebooks.
- 4 **Switch to a large self-configured notebook** if available.
- 5 **If not, manually set** executor memory, instances, cores, and driver memory.
- 6 **Test with a smaller subset** first to validate the configuration.
- 7 **Run the full workload** and monitor performance.
- 8 **Iterate** — hit and trial until stable and fast enough.

Manual Configuration: What to Change

```
1 spark = (SparkSession.builder
2   .appName("FixedLargeNotebook")
3   # Increase executor memory (e.g., 16g -> 32g)
4   .config("spark.executor.memory", "32g")
5   # Increase number of executors
6   .config("spark.executor.instances", "20")
7   # Increase cores per executor
8   .config("spark.executor.cores", "8")
9   # Increase driver memory if driver is collecting data
10  .config("spark.driver.memory", "16g")
11  # Tune shuffle partitions for large data
12  .config("spark.sql.shuffle.partitions", "400")
13  # Enable adaptive query execution (Spark 3+)
14  .config("spark.sql.adaptive.enabled", "true")
15  .getOrCreate())
```

Listing 3: Manually tuning Spark for a large job

- Start with values from old notebooks that worked.
- Adjust based on cluster capacity and job behavior.

Hit and Trial: What to Monitor

- **Spark UI** — check stages, tasks, and spills.
- **Executor memory usage** — are executors running out of memory?
- **Task duration** — which stages are slow?
- **Shuffle read/write** — excessive shuffle indicates partitioning issues.
- **GC time** — high garbage collection means memory pressure.

Iterative Tuning

- Change one parameter at a time.
- Document what you changed and the result.
- Repeat until the job completes in acceptable time.

Summary

- The notebook **times out** because default Spark resources are insufficient.
- Old notebooks solved this by **manually configuring** Spark.
- The tech team now provides **self-configured large notebooks**.
- **Preferred approach:** use a large notebook profile.
- **Fallback:** manually set executor memory, instances, cores, and driver memory.
- Expect **hit and trial** — tuning is iterative.

Key Takeaway

Match the Spark configuration to the workload.

Use large self-configured notebooks when available; otherwise, tune manually.

Best Practices

- 1 **Know your workload** — small, medium, or large.
- 2 **Prefer self-configured notebooks** provided by the tech team.
- 3 **Document** any manual configuration you apply.
- 4 **Test with a subset** before running the full job.
- 5 **Monitor Spark UI** during execution.
- 6 **Iterate** — do not expect perfect settings on the first try.
- 7 **Communicate** with the tech team about cluster limits.

References and Further Reading

- Apache Spark Documentation:
<https://spark.apache.org/docs/latest/configuration.html>
- Spark Tuning Guide: <https://spark.apache.org/docs/latest/tuning.html>
- Internal tech team documentation on notebook profiles.
- Old notebooks in the repository (3–5 years old) — examples of manual tuning.

Questions?